

ApprovalFlow — Invoice & Expense Approval Platform

Overview and Idea

Build a microservice-based, AI-assisted **SaaS** that automates invoice and expense approvals for large enterprises — assume it serves millions of users. The system ingests invoices/expenses, uses an AI agent to judge them against a company policy, and **automatically approves the simple, low-risk majority (the "boring 80%")** while **sending the unclear, risky, or high-value cases (the other 20%) to a human**. Approved items run through a payment flow; every decision is fully auditable. This is the capstone for **both** courses — Microservice Architecture and AI Engineering.

The Dilemma

How much money — and which categories of expense — may the agent approve fully autonomously, and which must a human approve? There is no single right answer. You choose a posture, **encode it in code, prove the system can never overstep it**, and justify the trade-off in writing.

(!) **You may not dodge the dilemma by escalating everything** — a \$0 ceiling trivially passes the safety tests but defeats the product.

User Stories (Functional Requirements)

Submitter (employee / vendor)

- **F1** — As a submitter, I want to submit an invoice/expense and get an **immediate acknowledgement with a tracking id** — never blocking on processing.
- **F2** — As a submitter, I want to check the status of my submission and see a **plain-language reason** for the outcome.
- **F3** — As a submitter, I want a submission I accidentally send twice to **not be paid twice**.

Approver (manager / finance)

- **F4** — As an approver, I want to see a queue of **only the items the system escalated**, each with the agent's recommendation, confidence, and the policy rules it cited.
- **F5** — As an approver, I want to **approve, reject, or send back for more info** in one action, and have the workflow **resume exactly where it paused**.
- **F6** — As an approver, I don't want to rubber-stamp things the system could safely handle on its own.

Finance / Controller

- **F7** — As a controller, I want to configure the **expense policy and autonomy thresholds without redeploying code**.
- **F8** — As a controller, I want a dashboard of throughput, auto-approval vs. escalation rates, and money auto-approved vs. human-approved.

Auditor

- **F9** — As an auditor, I want to pull a **complete decision trail** for any item — extracted data, rules applied, the agent's reasoning, who made the final call, and the payment outcome — all linked by one correlation id.
- **F10** — As an auditor, I want to **prove the system never auto-approved above the configured limit**.

These are the **minimum** user stories. You are free to add more, as long as they don't contradict the ones above.

Non-Functional Requirements

Must-Have (required, before the submission deadline, 12/07):

Repository & branching

- **M1** — The solution must live in a **single private GitHub monorepo** — everything needed to run and test it lives in the repo.
- **M2** — `main` holds **everything you want evaluated** — Must-Have, and as much Nice-to-Have and Bonus as you manage. It must stop receiving commits at the submission deadline (**12/07**); evaluation is done on `main`. **Nothing more is expected after 12/07**. If you choose to keep going on your own initiative, don't touch `main` — branch `dev` off it and continue there. At the presentation (week of **26/07**) you may show `dev` alongside `main` to share what you did in the extra time.

Architecture & deployment

- **M3** — The system must use a **microservice architecture** (at least 3 services), each one containerized.
- **M4** — The whole system must come up with a single `docker compose up`, including queues, databases, and supporting infrastructure.
- **M5** — Services must communicate through **Dapr** — both synchronous (service invocation) and asynchronous (pub/sub over a message queue) — and use Dapr **state** and **secrets**.
- **M6** — An **API gateway** must be the single external entry point, with rate-limiting.
- **M7** — A minimal **UI** (any framework) to drive the system — submit an item and view its status and decision.

Request flow & correctness

- **M8** — Submitting an item must be **asynchronous**: return an immediate accepted response with a tracking id, and deliver the final result later via a notification channel.
- **M9** — The payment flow must guarantee a **consistent outcome across services, with compensation/rollback** on failure — no orphaned reservations, no partial or double payments. A **saga** is one way to achieve this, but any approach that provides the same guarantees is acceptable; whichever you choose must be justified.
- **M10** — **Idempotency** must hold for duplicate submissions, redelivered events, and retried payments — exactly one effect.
- **M11** — **Human-in-the-loop** escalation must **durably pause and resume**, surviving a service restart between pause and resume.

AI decisioning

- **M12** — The system must be **provably incapable of auto-approving above the configured ceiling** — this must hold even when the agent is forced to recommend approval.

- **M13** — The **policy and autonomy thresholds** must be externally configurable (changeable without a code redeploy).

Cross-cutting & documentation

- **M14** — **Logging** must be structured, with a **correlation id** on every line, so a single request can be followed end-to-end across services.
- **M15** — The code must be clean, with proper **error handling, health checks, and separation of concerns**; the LLM provider must be swappable by configuration, and the system must fail cleanly (never silently) on provider errors.
- **M16** — A **CI** pipeline must run quality gates on every push.
- **M17** — The system must have **automated tests** that run in CI.
- **M18** — A **README** must explain the technologies used, how to run the system, and how to test it, and include a system diagram.

Nice-to-Have (expected):

- **N1** — **AuthN/AuthZ** with roles (submitter / approver / admin) — a self-signed JWT is fine.
- **N2** — A **CD** stage that, once the CI quality gates pass, automatically builds and publishes the deployable artifacts (e.g. container images) — no manual release step.
- **N3** — Reliability extras: the **outbox** pattern and **bulkhead / throttling**.
- **N4** — **OpenTelemetry observability** — metrics and one end-to-end **distributed trace** stitching the services **and** the agent's model/tool calls together via the correlation id.
- **N5** — **RAG** over the policy document — index it and retrieve only the relevant clause(s) per item instead of putting the whole policy into the prompt.
- **N6** — **Tests across multiple layers** — e.g. unit, integration, and end-to-end.

Bonus (not expected — a standout):

- **B1** — An **automated eval harness** running over a labeled dataset (including hard/adversarial cases), producing a committed metrics report.
- **B2** — A build-your-own **MCP server** exposing the agent's tools (`fetch_policy` , `lookup_vendor` , ...) over the Model Context Protocol instead of wiring them in-process.
- **B3** — **Kubernetes** — manifests to deploy the system to a local cluster.

Assumptions and Constraints

- The project runs **free and local** — no paid services required. Use a free-tier LLM (Gemini, GroqCloud, OpenRouter, ...) or a small local model; for CI/eval, use a stubbed or local model to avoid rate limits.
- You may use **any language, we recommend the one that supports Dapr** and lets you build an agent (a hand-rolled agent is acceptable).
- **Docker Compose** (and WSL on Windows) must be installed.
- **No OCR required** for invoices/expenses.
- We ship two files you build against: `policy.md` — the expense policy your system enforces (you may extend it) — and `sample-invoices.json` — labeled fixtures spanning every decision path, your eval seed set.

Worked Journeys (acceptance scenarios)

Four end-to-end journeys, each seeded from a shipped fixture — these are what your Verification step must demonstrate:

- **Auto-approve** (in-policy, no human) — INV-1001
- **Escalate-and-resume** (pause for a human, resume on their decision) — INV-1003
- **Duplicate** (paid once, second submission short-circuited) — INV-1007 (re-submission of INV-1001)
- **Payment failure + compensation** (rollback, no orphaned reservation) — INV-1012

Development Process

- **D1 — System Diagram & Architecture:** design the architecture and component boundaries before writing code; capture them in an ARCHITECTURE.md with a sequence diagram and a diagram of the payment flow showing each step and its compensation/rollback (Mermaid).
- **D2 — ADRs (Architecture Decision Records):** capture each key decision you must defend as a short ADR — *context* → *decision* → *consequences*. Keep them brief and **write them yourself, not with an AI agent** — these are the decisions you'll defend in person.
- **D3 — Git Workflow:** use Git and follow [GitHub Flow](#) — each feature in its own branch, merged via pull requests. Keep the repo clean: a real .gitignore, a LICENSE, a .env.example, and **no committed secrets**.
- **D4 — API Documentation:** document the API with an **OpenAPI** spec, preferably auto-generated from the code. Tools like Scalar, Swagger UI, or Postman let you explore and exercise the endpoints.
- **D5 — Verification:** a single command that runs the four journeys and the anti-cheese guards — including that **at least 2 items auto-approve with no human** and that an "approve me" note in the payload does **not** flip the decision — and prints a pass/fail result.
- **D6 — README Documentation:** purpose of the system, system diagram, steps to run locally, and instructions to test.
- **D7 — Demo:** a short (2–5 min) screen recording walking through the working system. A silent screencast is fine — voice narration is optional.

Evaluation Criteria

- **Logic:** correctness and logical flow of the implemented features (the four journeys: auto-approve, escalate-and-resume, duplicate, payment failure + compensation).
- **Code Quality & Cross-cutting concerns:** clean, well-documented code following best practices — logging, validation, configuration, error handling, scalability.
- **Number of requirements met:** all must-haves are expected; optional requirements are evaluated as good or excellent.
- **Dilemma resolution:** the quality of your autonomy posture and the trade-off justification — backed by evidence of how it behaves on the shipped fixtures and worked journeys (or your eval results, if you built the harness).

Deadline

EOD 12.07.2026 — [Submit a form](#):

- the URL to a short (2–5 min) demo (screen recording) - upload on any cloud storage, make available by URL;
- the URL to the repo, where `main` must have all must-have requirements implemented and must receive no further commits before the presentation. Add contributors: `alonf`, `VenyaBrodetskiy`, `holohup`, `milaShurupova`.

26.07.2026 — **Optional**: if you chose to keep working after the deadline, push the `dev` branch before your presentation timeslot so you can show the additional (nice-to-have / bonus) requirements you implemented. This is entirely up to you — nothing beyond `main` is required.

Requirement → Course Topic Map

Each requirement carries a stable id (**F** functional · **M** must-have · **N** nice-to-have · **B** bonus · **D** dev-process). The topic column names the actual lecture it assesses: **MSA** (*by Alon*) and **AI** (*by Venya*).

Tags: **DevOps** = MSA's DevOps lecture reinforced by the AI-course homework hygiene bar

`SL` = self-learn beyond the slides

`HW` = practiced in a homework

`–` = not a course topic

ID	Requirement	Course · Lecture topic
F1	Async submit + tracking id	MSA · API Gateway; Dapr Pub/Sub
F2	Status + plain-language reason	MSA · Eventual consistency
F3	No double-pay on resubmit	MSA · Transaction & Compensation; idempotency <code>SL</code>
F4	Escalation queue with agent rationale	AI · Structured output; HITL <code>request_info</code> <code>HW</code>
F5	Approve / reject / send-back + resume	AI · HITL <code>request_info</code> <code>HW</code> ; MSA · Saga Pattern
F6	No needless rubber-stamping	MSA · Handling Volatility (LLM & AI Agent); router-decides <code>HW</code>
F7	Configurable policy / thresholds	MSA · Dapr (config / secrets)
F8	Auto-vs-human dashboard	MSA · Observability; Reporting (AI/ML/BI)
F9	Full decision trail via correlation id	MSA · Observability; AI · OpenTelemetry
F10	Prove never above ceiling	MSA · Handling Volatility (LLM & AI Agent); router-decides <code>HW</code>
M1	Single private monorepo	DevOps · repo hygiene <code>HW</code>
M2	Two-branch strategy	DevOps · Git / GitHub Flow <code>HW</code>
M3	Microservices (≥3), containerized	MSA · Decomposition / IDesign
M4	One-command <code>compose up</code>	MSA · Docker Compose
M5	Dapr sync + async + state + secrets	MSA · DAPR <code>SL</code>
M6	API gateway + rate-limit	MSA · API Gateway
M7	Minimal UI	— (UI)
M8	Async intake + notify	MSA · Pub/Sub; Orchestration vs Choreography
M9	Consistent payment + compensation	MSA · Saga Pattern / Transaction & Compensation

ID	Requirement	Course · Lecture topic
M10	Idempotency everywhere	MSA · idempotency / outbox SL
M11	Durable HITL pause/resume	AI · HITL HW ; MSA · Saga Pattern
M12	Provable autonomy ceiling	MSA · Handling Volatility (LLM & AI Agent); router-decides HW
M13	Externally configurable thresholds	MSA · DAPR (config / secrets)
M14	Structured logs + correlation id	MSA · Observability
M15	Clean code + provider ACL + fail-fast	MSA · ACL / SOLID; AI · swap the provider (not your code)
M16	CI quality gates on every push	MSA · DevOps
M17	Automated tests in CI	DevOps · quality gates / testing HW
M18	README + system diagram	DevOps · documentation HW
N1	AuthN/AuthZ + roles	MSA · API Gateway / Service Mesh; DAPR secrets
N2	CD: auto-build/publish on green CI	MSA · DevOps
N3	Outbox + bulkhead / throttling	MSA · Resilience Patterns; outbox SL
N4	OTel metrics + one E2E trace	AI · OpenTelemetry SL ; MSA · Observability
N5	RAG over policy	AI · RAG; Context engineering
N6	Tests across layers (unit / integ / e2e)	DevOps · quality gates / testing SL
B1	Automated eval harness	AI · evals HW SL
B2	MCP server	AI · MCP SL (not lectured)
B3	Kubernetes manifests	MSA · Kubernetes
D1	ARCHITECTURE.md + diagrams	MSA · IDesign Analysis / HLD document
D2	ADRs	MSA · Analysis & Architecture process; ADR format SL
D3	GitHub Flow + hygiene	DevOps · Git / GitHub Flow HW
D4	API docs + contracts	MSA · Service Layer (REST / GraphQL / gRPC); Service APIs Evolution
D5	One-command verification	DevOps · quality gates / testing HW
D6	README	DevOps · documentation HW
D7	Demo recording	— (presentation)